EAST WEST UNIVERSITY

Lab 5

DDA (Digital Differential Analyzer) Algorithm

Course Title: *Computer Graphics*

Course Code: *CSE 420*

Semester: *Spring 2026*

Section: *01*

Submitted by

Name: *Shairin Akter Hashi*
ID: *2022-2-60-102*
Date: *16/02/2026*

Submitted to

Dr. Md. Tauhid Bin Iqbal

Assistant Professor

Department of Computer Science and Engineering

East West University

Lab Task

```
#include<stdio.h>
#include <GL/gl.h>
#include <GL/glut.h>
#include <math.h>

float X,Y,X1, Y1, X2, Y2, m;
float dx, dy;

// Function to display the DDA line
void display(void)
{
  glClear(GL_COLOR_BUFFER_BIT);
  glColor3f(1.0, 1.0, 1.0);
  glLineWidth(20.0f);
  glPointSize(10.0f);
 // Calculate differences and slope
  dx = X2 - X1;
  dy = Y2 - Y1;
  m = dy / dx;

  glBegin(GL_POINTS);

  // Calculate slope
  float dx = X2 - X1;
  float dy = Y2 - Y1;
  float m;

  if (dx != 0)
    m = dy / dx;

  /* =========================
    🟩 VERTICAL LINES
  ========================= */

  // Positive Vertical (Bottom → Top)
  if (dx == 0 && Y1 <= Y2)
  {
    while (Y1 <= Y2)
    {
      X = roundf(X1);
      Y = roundf(Y1);

      glVertex2f(X / 100, Y / 100);
      printf("(%.2f ,%.2f) ----> Rounded ----> (%.2f ,%.2f)\n", X1,Y1,X,Y);

      Y1 = Y1 + 1;
```

```
    }
  }

  // Negative Vertical (Top → Bottom)
  else if (dx == 0 && Y1 > Y2)
  {
    while (Y1 >= Y2)
    {
      X = roundf(X1);
      Y = roundf(Y1);

      glVertex2f(X / 100, Y / 100);
      printf("(%.2f ,%.2f) ----> Rounded ----> (%.2f ,%.2f)\n", X1,Y1,X,Y);

      Y1 = Y1 - 1;
    }
  }

  /* ========================
     🟩 HORIZONTAL LINES
  ======================== */

  // Positive Horizontal (Left → Right)
  else if (m == 0 && X1 <= X2)
  {
    while (X1 <= X2)
    {
      X = roundf(X1);
      Y = roundf(Y1);

      glVertex2f(X / 100, Y / 100);
      printf("(%.2f ,%.2f) ----> Rounded ----> (%.2f ,%.2f)\n", X1,Y1,X,Y);

      X1 = X1 + 1;
    }
  }

  // Negative Horizontal (Right → Left)
  else if (m == 0 && X1 > X2)
  {
    while (X1 >= X2)
    {
      X = roundf(X1);
      Y = roundf(Y1);

      glVertex2f(X / 100, Y / 100);
      printf("(%.2f ,%.2f) ----> Rounded ----> (%.2f ,%.2f)\n", X1,Y1,X,Y);

      X1 = X1 - 1;
    }
```

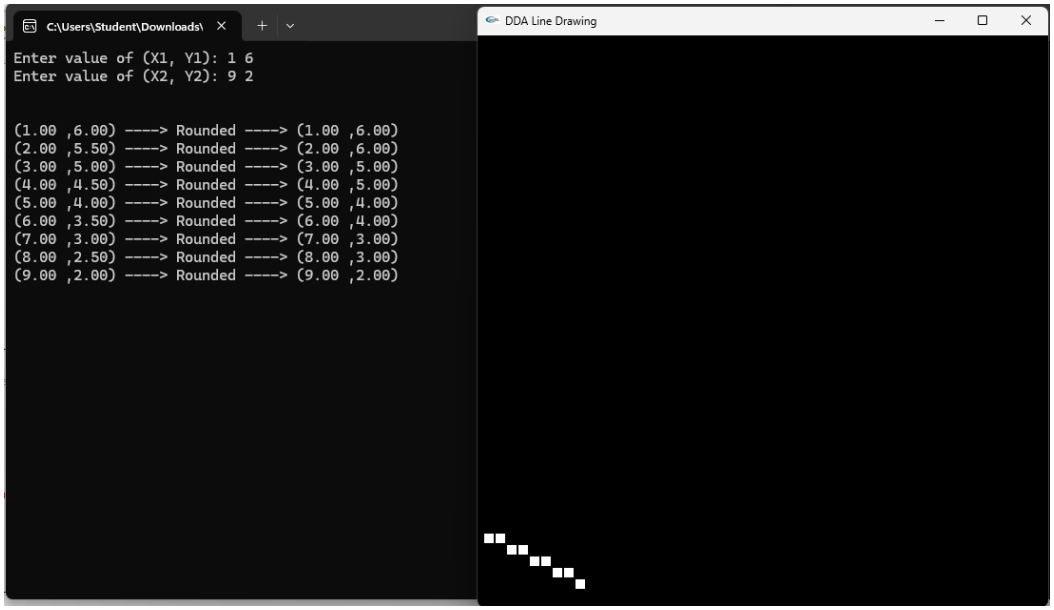
```
  }

  /* ========================
     🟩 POSITIVE SLOPE
  ======================== */

  // 0 < m < 1
  else if (m > 0 && m < 1)
  {
     while (X1 <= X2)
     {
        X = roundf(X1);
        Y = roundf(Y1);

        glVertex2f(X / 100, Y / 100);
        printf("(%.2f ,%.2f) ----> Rounded ----> (%.2f ,%.2f)\n", X1,Y1,X,Y);

        X1 = X1 + 1;
        Y1 = Y1 + m;
     }
  }

  // m == 1
  else if (m == 1)
  {
     while (X1 <= X2 && Y1 <= Y2)
     {
        X = roundf(X1);
        Y = roundf(Y1);

        glVertex2f(X / 100, Y / 100);
        printf("(%.2f ,%.2f) ----> Rounded ----> (%.2f ,%.2f)\n", X1,Y1,X,Y);

        X1 = X1 + 1;
        Y1 = Y1 + 1;
     }
  }

  // m > 1
  else if (m > 1)
  {
     while (Y1 <= Y2)
     {
        X = roundf(X1);
        Y = roundf(Y1);

        glVertex2f(X / 100, Y / 100);
        printf("(%.2f ,%.2f) ----> Rounded ----> (%.2f ,%.2f)\n", X1,Y1,X,Y);
```

```
      X1 = X1 + (1.0f / m);
      Y1 = Y1 + 1;
    }
  }

  /* ========================
     🟩 NEGATIVE SLOPE
  ========================= */

  // -1 < m < 0
  else if (m < 0 && m > -1)
  {
    while (X1 <= X2)
    {
      X = roundf(X1);
      Y = roundf(Y1);

      glVertex2f(X / 100, Y / 100);
      printf("(%.2f ,%.2f) ----> Rounded ----> (%.2f ,%.2f)\n", X1,Y1,X,Y);

      X1 = X1 + 1;
      Y1 = Y1 + m;   // m negative
    }
  }

  // m == -1
  else if (m == -1)
  {
    while (X1 <= X2 && Y1 >= Y2)
    {
      X = roundf(X1);
      Y = roundf(Y1);

      glVertex2f(X / 100, Y / 100);
      printf("(%.2f ,%.2f) ----> Rounded ----> (%.2f ,%.2f)\n", X1,Y1,X,Y);

      X1 = X1 + 1;
      Y1 = Y1 - 1;
    }
  }

  // m < -1
  else if (m < -1)
  {
    while (Y1 >= Y2)
    {
      X = roundf(X1);
      Y = roundf(Y1);
```

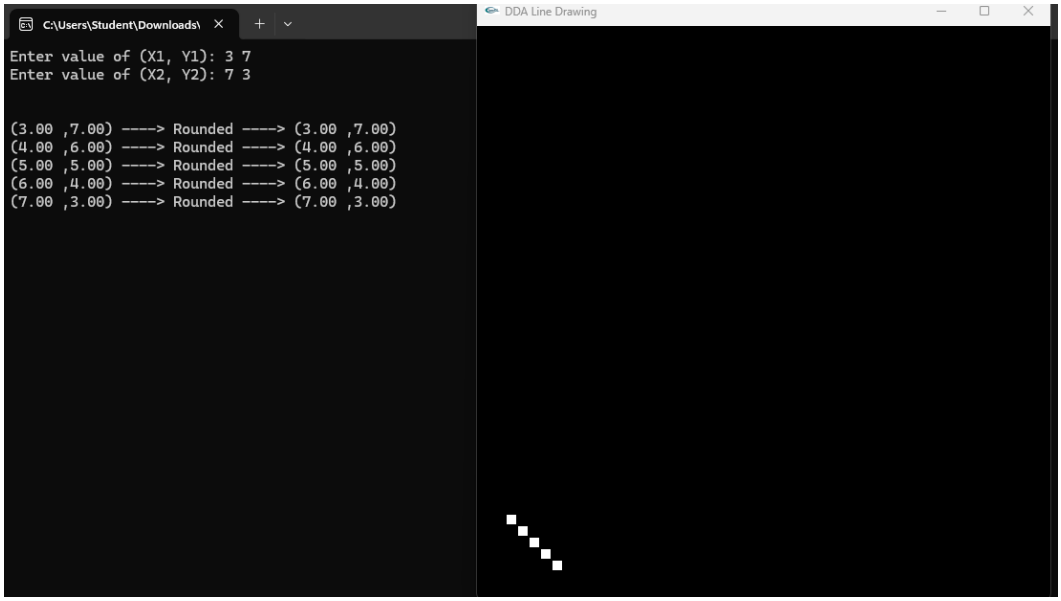
```
        glVertex2f(X / 100, Y / 100);
        printf("(%.2f ,%.2f) ----> Rounded ----> (%.2f ,%.2f)\n", X1,Y1,X,Y);

        X1 = X1 + (1.0f / m);   // small negative
        Y1 = Y1 - 1;
      }
    }

    glEnd();

    glFlush();
}


// Function to initialize OpenGL settings
void init(void)
{
    glClearColor(0.0, 0.0, 0.0, 0.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(0.0, 0.5 , 0.0, 0.5, -1.0, 1.0);
}

// Main function
int main(int argc, char** argv)
{
    // Input the coordinates for the line
    printf("Enter value of (X1, Y1): ");
    scanf("%f %f", &X1, &Y1);
    printf("Enter value of (X2, Y2): ");
    scanf("%f %f", &X2, &Y2);
    printf("\n \n");


    // Initialize GLUT and create a window
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("DDA Line Drawing");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

Output of Lab Task

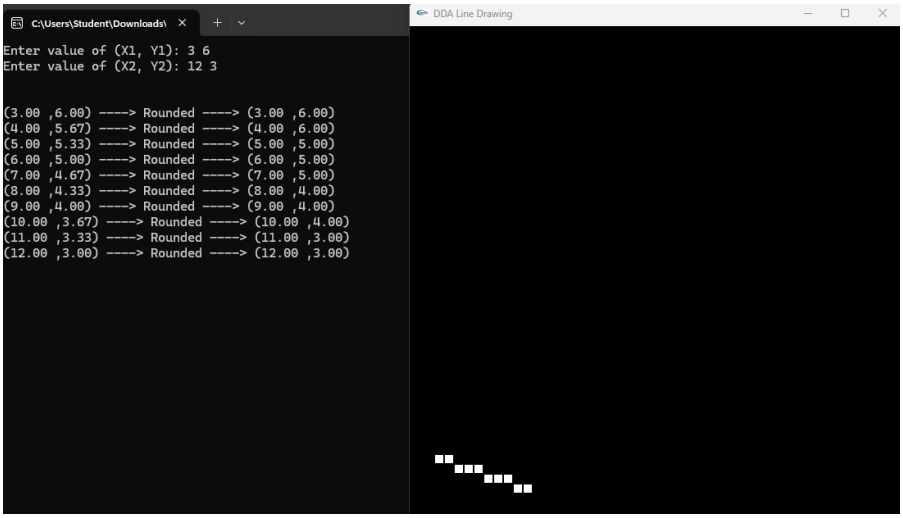
Case 1: -1 < m < 0



Case 2: m = -1



Case 1: m < -1


C:\Users\Student\Downloads\
Enter value of (X1, Y1): 3 6
Enter value of (X2, Y2): 12 3
(3.00 ,6.00) ----> Rounded ----> (3.00 ,6.00)
(4.00 ,5.67) ----> Rounded ----> (4.00 ,6.00)
(5.00 ,5.33) ----> Rounded ----> (5.00 ,5.00)
(6.00 ,5.00) ----> Rounded ----> (6.00 ,5.00)
(7.00 ,4.67) ----> Rounded ----> (7.00 ,5.00)
(8.00 ,4.33) ----> Rounded ----> (8.00 ,4.00)
(9.00 ,4.00) ----> Rounded ----> (9.00 ,4.00)
(10.00 ,3.67) ----> Rounded ----> (10.00 ,4.00)
(11.00 ,3.33) ----> Rounded ----> (11.00 ,3.00)
(12.00 ,3.00) ----> Rounded ----> (12.00 ,3.00)
DDA Line Drawing